```
!pip install pandas numpy matplotlib seaborn scikit-learn
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/di
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/p
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.1
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dis
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/pyt
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving archive.zip to archive (2).zip
```

```
import zipfile

with zipfile.ZipFile("train_FD001.txt.zip", 'r') as zip_ref:
    zip_ref.extractall()

print("Done extracting FD001!")
```

```
Done extracting FD001!
```

```
import os
print(os.listdir())
```

```
['.config', 'archive.zip', 'CMaps', 'archive (1).zip', 'archive (2).zip',
```

```
import os

for root, dirs, files in os.walk('.'):
```

```
    for file in files:
        if file.endswith('.txt'):
            print(os.path.join(root, file))
```

```
./CMaps/readme.txt
./CMaps/RUL_FD004.txt
./CMaps/RUL_FD003.txt
./CMaps/test_FD001.txt
./CMaps/train_FD001.txt
./CMaps/train_FD002.txt
./CMaps/train_FD004.txt
./CMaps/test_FD002.txt
./CMaps/test_FD003.txt
./CMaps/RUL_FD002.txt
./CMaps/train_FD003.txt
./CMaps/test_FD004.txt
./CMaps/x.txt
./CMaps/RUL_FD001.txt
```

```
df = pd.read_csv("CMaps/train_FD001.txt", sep=" ", header=None)
```

```
import os
print(os.listdir())
```

```
['.config', 'archive.zip', 'CMaps', 'archive (1).zip', 'archive (2).zip',
```

```
import os

print(os.listdir("CMaps"))
```

```
['Damage Propagation Modeling.pdf', 'readme.txt', 'RUL_FD004.txt', 'RUL_F
```

```
import pandas as pd

df = pd.read_csv("CMaps/train_FD001.txt", sep=" ", header=None)
df.head()
```

	0	1	2	3	4	5	6	7	8	9	...	
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	8138.
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	8131.
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	8133.
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	8133.
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	8133.

5 rows × 28 columns

```
df = df.iloc[:, :26]
df.head()
```

	0	1	2	3	4	5	6	7	8	9	...	1
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.6
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.2
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	522.4
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	522.8
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	522.1

5 rows × 26 columns

```
columns = ['unit_number','time_cycle'] + \
          [f'op_setting_{i}' for i in range(1,4)] + \
          [f'sensor_{i}' for i in range(1,22)]

df.columns = columns

df.head()
```

	unit_number	time_cycle	op_setting_1	op_setting_2	op_setting_3	sens
0	1	1	-0.0007	-0.0004	100.0	5
1	1	2	0.0019	-0.0003	100.0	5
2	1	3	-0.0043	0.0003	100.0	5
3	1	4	0.0007	0.0000	100.0	5
4	1	5	-0.0019	-0.0002	100.0	5

5 rows × 26 columns

```
# Get max cycle for each engine
rul = df.groupby('unit_number')['time_cycle'].max().reset_index()
rul.columns = ['unit_number', 'max_cycle']

# Merge with original data
df = df.merge(rul, on='unit_number')

# Calculate RUL
df['RUL'] = df['max_cycle'] - df['time_cycle']

df.head()
```

	unit_number	time_cycle	op_setting_1	op_setting_2	op_setting_3	sens
0	1	1	-0.0007	-0.0004	100.0	5
1	1	2	0.0019	-0.0003	100.0	5
2	1	3	-0.0043	0.0003	100.0	5
3	1	4	0.0007	0.0000	100.0	5
4	1	5	-0.0019	-0.0002	100.0	5

5 rows × 28 columns

```
from sklearn.model_selection import train_test_split

# Use only sensor columns
features = [col for col in df.columns if 'sensor' in col]

X = df[features]
y = df['RUL']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=50)
model.fit(X_train, y_train)
```

```
▾ RandomForestRegressor
RandomForestRegressor(n_estimators=50)
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)
print("MAE:", mae)
```

```
MAE: 29.389159195541556
```

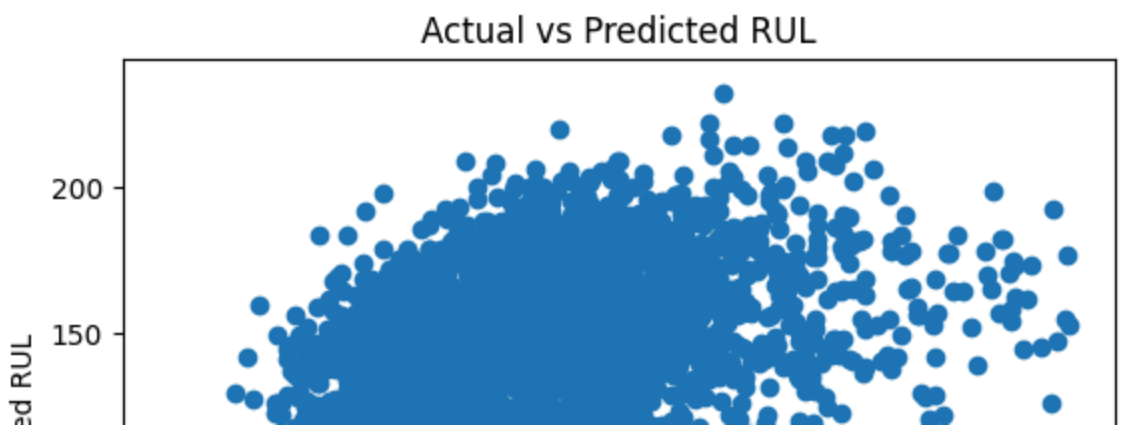
```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, max_depth=10)
model.fit(X_train, y_train)
```

```
▾ RandomForestRegressor
RandomForestRegressor(max_depth=10)
```

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel("Actual RUL")
plt.ylabel("Predicted RUL")
plt.title("Actual vs Predicted RUL")
plt.show()
```




```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.01)
df['anomaly'] = iso.fit_predict(df[features])

df['anomaly'].value_counts()
```

anomaly	count
1	20424
-1	207

dtype: int64